



MÓDULO 01: FUNDAMENTOS DA LINGUAGEM

Fundamentos da Linguagem Python

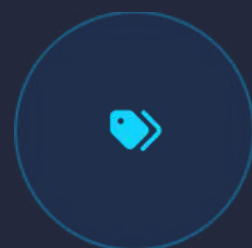
Transformando conceitos abstratos em
modelos mentais claros para a
iniciarmos nossa jornada no Python!

Instrutora: Luiza Vieira (vbluuiza)



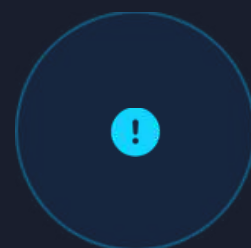


SUMÁRIO DO MÓDULO



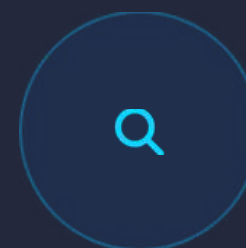
**Conheça a
Linguagem
Python**

AULA 01



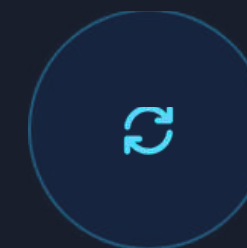
**Variáveis, Sintaxe
e Tipos Primitivos**

AULA 02



**Estruturas de
Decisão**

AULA 03



**Estruturas de
Repetição**



Conheça a história do Python





O Conceito de Variáveis



Imagine uma **mudança**: etiquetas identificam o conteúdo de cada caixa para facilitar a organização.



No Python, a **variável é a etiqueta** que aponta para um valor guardado dentro de uma "caixa".



Variável é um **nome amigável** que referencia um endereço específico na memória RAM do computador.

 **idade**



25

CONTEÚDO DA MEMÓRIA



Atribuição de Valores

O significado de "="

- ↔ O símbolo = significa **ATRIBUIÇÃO**, não igualdade matemática.
- 👉 Diz ao Python: "Pegue o valor à direita e dê o nome à esquerda para ele".
- 📁 O valor é guardado na memória e a "etiqueta" aponta para esse endereço.

Estrutura da Linha

- 🏷 Lado Esquerdo: O nome da variável (a etiqueta).
- 📦 Lado Direito: O dado ou valor (o conteúdo).



```
# Atribuindo o valor à variável  
idade = 25  
nome = "Python"
```



Palavras reservadas no Python

As palavras reservadas do Python são termos que definem a sintaxe e estrutura da linguagem e não podem ser usadas como nomes de variáveis, funções ou classes

Podemos usar o comando `help("keywords")` no interpretador do Python para visualizar todas as palavras reservadas da linguagem.

```
• >>> help("keywords")
```

```
Here is a list of the Python keywords. Enter any keyword to get more help.
```

```
False  
None  
True  
and  
as  
assert  
async  
await  
break
```

```
class  
continue  
def  
del  
elif  
else  
except  
finally  
for
```

```
from  
global  
if  
import  
in  
is  
lambda  
nonlocal  
not
```

```
or  
pass  
raise  
return  
try  
while  
with  
yield
```



Tipos Primitivos: str e int

“ str (String)

**Representa textos e sequências de caracteres.
Deve estar sempre entre aspas simples ou duplas.**

int (Integer)

Representa números inteiros, positivos ou negativos, sem nenhuma parte decimal.



```
# Exemplo de Atribuição
usuario="Luiza" # Tipo str
pontos = 100    # Tipo int

# Verificando os tipos
print(type(usuario))
print(type(pontos))
```



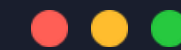

Tipos Primitivos: float e bool

float (Ponto Flutuante)

Representa números com casas decimais. Essencial para precisão em cálculos financeiros, pesos e medidas científicas.

bool (Booleano)

Estados lógicos binários fundamentais para decisões. Possui apenas dois valores possíveis: **True** ou **False**.



```
# Definindo decimais e lógicos
preco = 49.90
logado = True

# Verificando a categoria
print(type(preco))print(type(logado))
```

 MENTAL CHECK: USE TYPE() PARA VALIDAR



Identificação com a função `type()`



- 📄 **O "Scanner" do Python:** Revela a categoria exata do dado guardado na variável.
- ⚡ **Tipagem Dinâmica:** O Python descobre o tipo em tempo de execução, conforme o valor muda.
- ✳️ **Depuração:** Ferramenta essencial para entender por que um erro de tipo ocorreu.



```
usuario = "Luiza"
idade = 25

print(type(usuario)) # <class 'str'>
print(type(idade)) # <class 'int'>

# Funciona com valores diretos:
print(type(49.90)) # <class 'float'>
```




Sintaxe e Indentação

Leitura Sequencial

O Python lê o código como um livro: de cima para baixo, linha por linha.

Indentação Estrutural

Espaços no início da linha definem a quais blocos as instruções pertencem.

Case-Sensitivity

O Python diferencia letras maiúsculas de minúsculas (Ex: idade != Idade).



EXEMPLO CORRETO

```
if True:
    print("Identado com 4 espaços")
# O print faz parte do bloco if
```



EXEMPLO INCORRETO

```
if True:
print("Erro de Indentação!")
# O Python não sabe onde o bloco começa
```




Comentários e Saída de Dados

Comentários com

Notas escritas para humanos. O interpretador Python ignora completamente estas linhas durante a execução.

>_ Função print()

A ferramenta essencial para exibir resultados, estados de variáveis e mensagens diretamente no console/terminal.



```
# Documentando o código
usuario = "Ana"
# Exibindo o valor no terminal
print(usuario)
# Saída esperada: Ana
```




Boas Práticas de Nomenclatura

Padrão snake_case

Use letras minúsculas separadas por sublinhado (underline) para nomes compostos.

Restrições de Início

Proibido começar nomes com números ou usar espaços em branco entre caracteres.

Case-Sensitive

Python diferencia maiúsculas de minúsculas.
idade é diferente de **Idade**.



</> GUIA DE ESTILO (PEP 8)

CORRETO

```
nome_usuario = "Ana"  
total_vendas_2024 = 5000
```

ERRADO

```
nome usuario = "Ana" # Espaço  
1_lugar = "Ouro"     # Inicia com número
```

Dica: Nomes claros e descritivos facilitam a leitura do seu código por outros desenvolvedores.



Operadores

Principais Operadores no Python

Operadores são símbolos usados para fazer ações com valores e variáveis no Python. Eles permitem realizar cálculos, comparações e verificações lógicas no código.

Um mergulho nos booleanos

É um tipo de dado usado para representar valores lógicos: **True e False**.

Eles normalmente são o **resultado de expressões relacionais e lógicas**, como comparações (`==`, `>`, `<`) e operações com `and`, `or` e `not`.

Esse tipo é fundamental para controlar o fluxo do programa em estruturas condicionais e de repetição, permitindo que o código tome decisões com base em condições.

OPERADORES DE COMPARAÇÃO

<code>></code>	MAIOR
<code><</code>	MENOR
<code>>=</code>	MAIOR OU IGUAL
<code><=</code>	MENOR OU IGUAL
<code>==</code>	IGUAL A
<code>!=</code>	DIFERENTE DE

OPERADORES ARITMÉTICOS

<code>+</code> <code>-</code> <code>*</code> <code>/</code>	
<code>%</code>	MÓDULO
<code>+=</code> , <code>-=</code>	

OPERADORES LÓGICOS

<code>&</code>	AND	Retorna True apenas se ambos os operandos forem verdadeiros.
<code> </code>	OR	Retorna True se pelo menos um dos operandos forem verdadeiros.
<code>not</code>	NOT	Inverte o valor lógico do operando



Estruturas de Decisão: if, elif e else

if (Se)

Inicia a verificação. Se a condição for verdadeira, o bloco de código é executado.

elif (Senão, se)

Testa uma nova condição caso a anterior tenha falhado (resultado Falso).

else (Senão)

O "porto seguro". Executa se nenhuma das condições anteriores for atendida.



```
# Exemplo de lógica condicional
idade = 18

if idade >= 18:
    print("Acesso liberado")

elif idade == 17:
    print("Quase lá!")

else:
    print("Acesso negado")
```



Note a **indentação**: os espaços antes do print definem o que pertence a cada decisão.

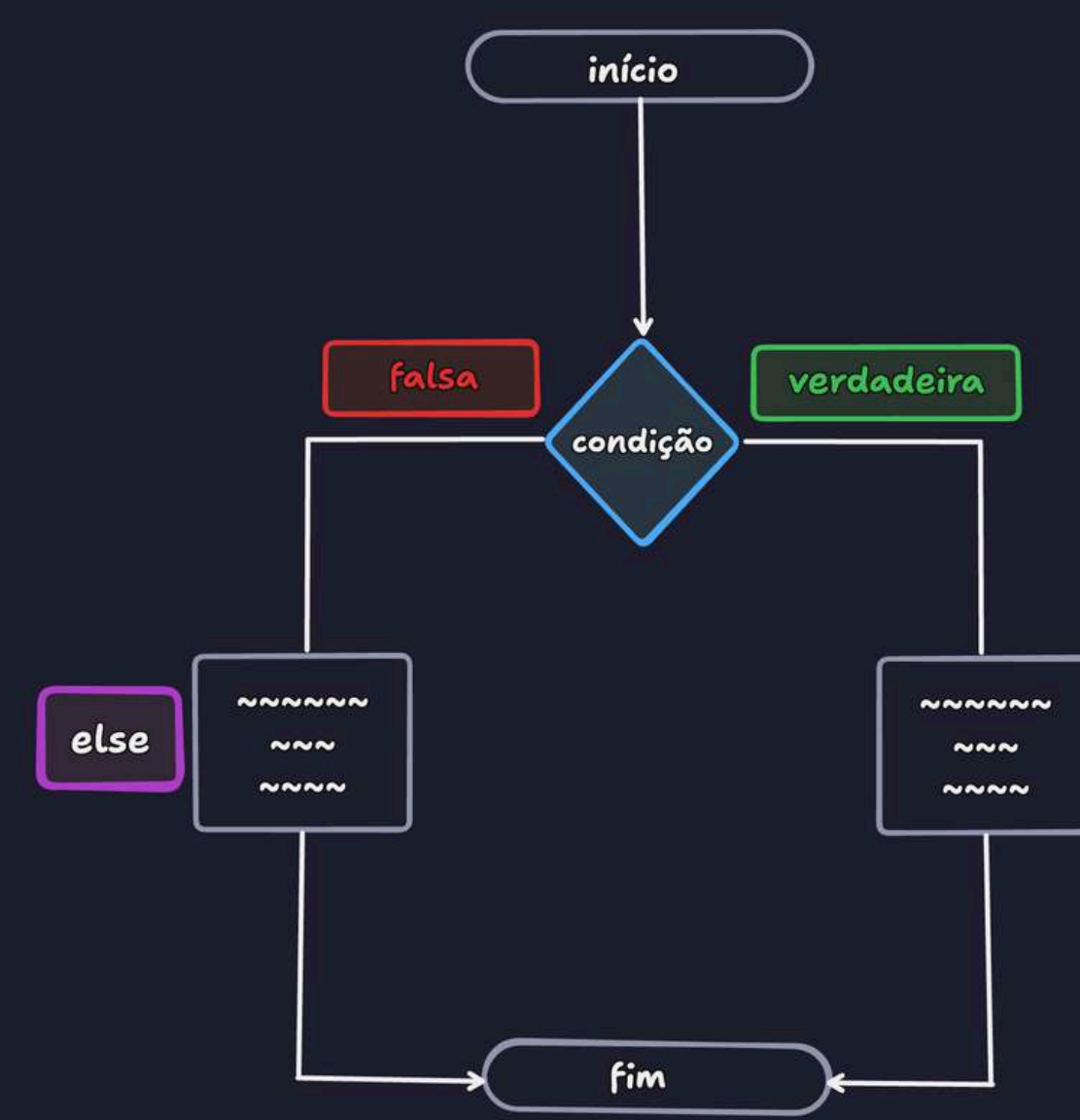


Fluxogramas

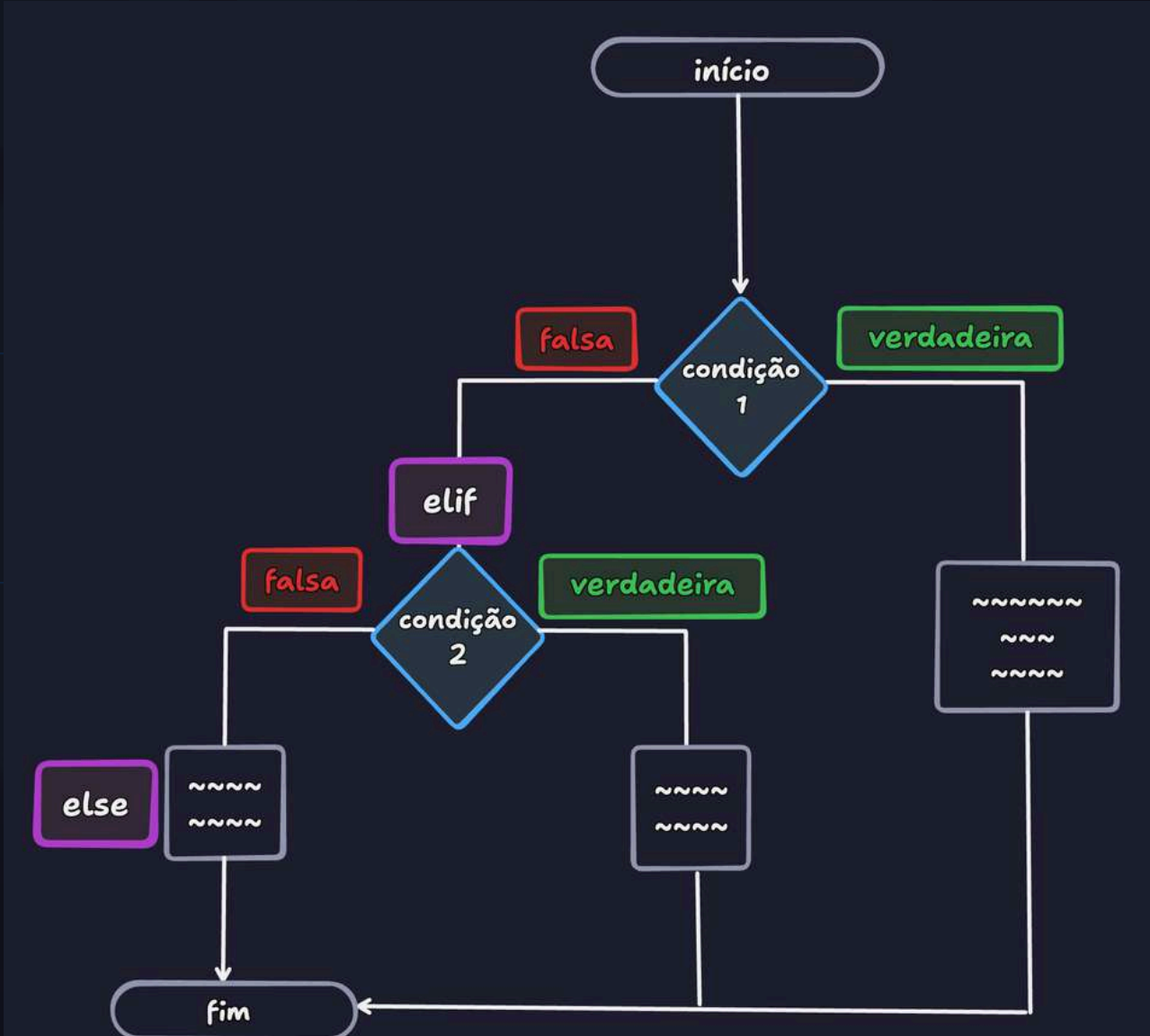
if



if-else

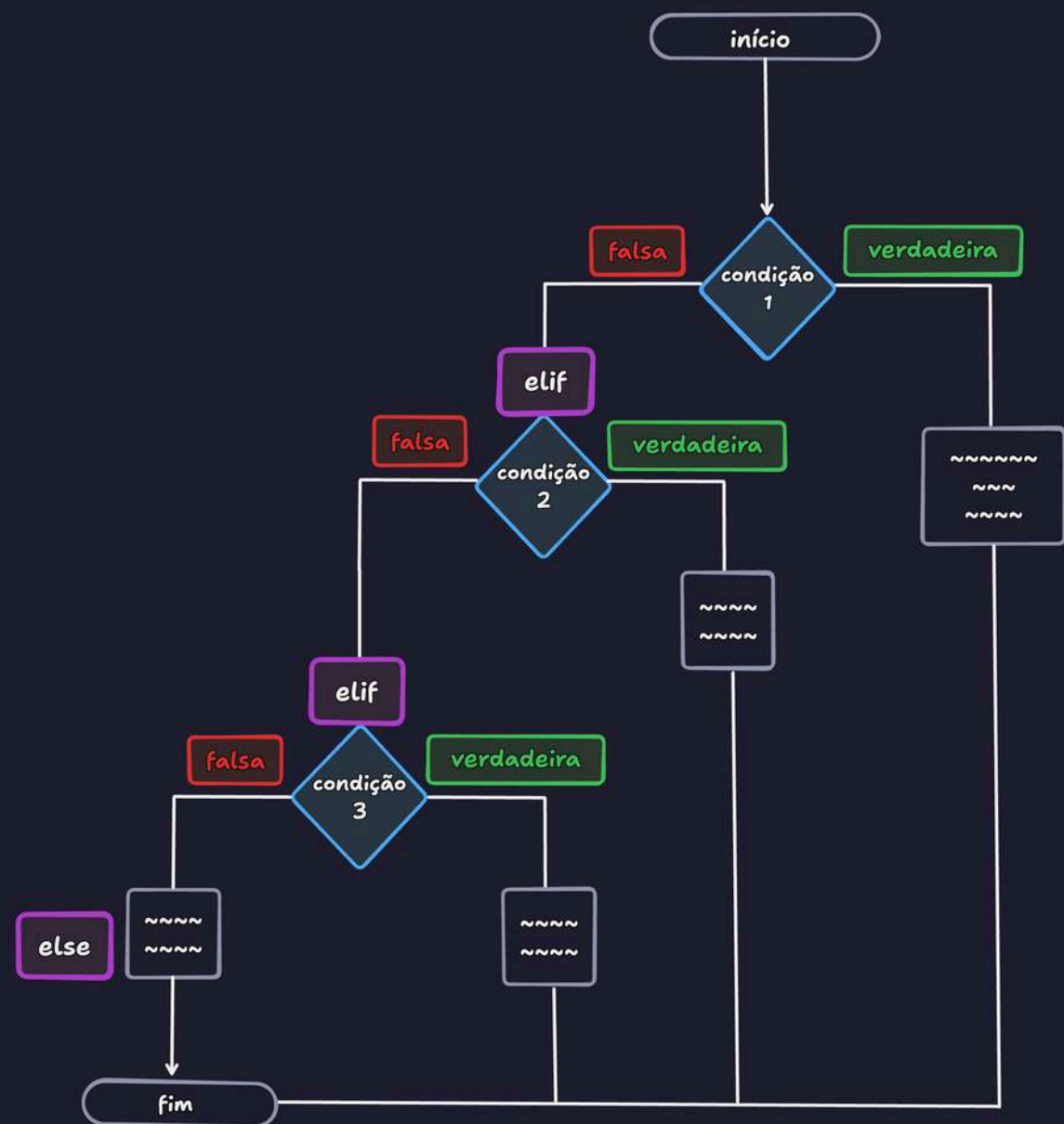


elif

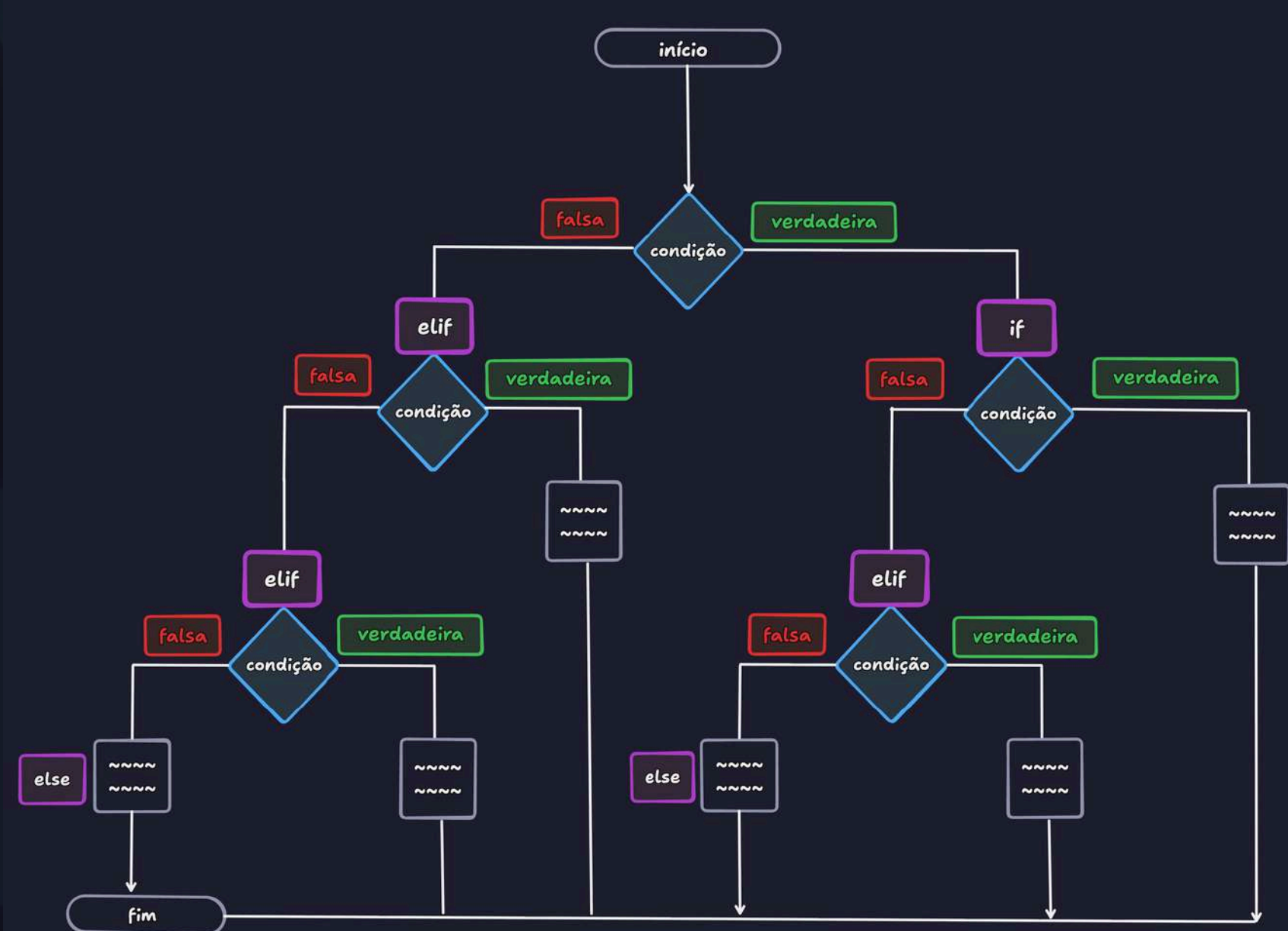


Fluxogramas

Múltiplos elif



if aninhado





Regras do Else

- ⊘ **O else significa “caso contrário”**
Ele executa quando a condição do if é falsa.
- ⊘ **O else não recebe condição**
O else nunca tem comparação na frente.
- ⊘ **O else precisa estar ligado a um if**
Não existe else sozinho.
- ⊘ **O else vem sempre no final da estrutura**
- ⊘ **Só pode existir um else por bloco de decisão**
Você pode ter vários elif, mas só um else.
- ⊘ **O else precisa de dois pontos :**
- ⊘ **O bloco do else precisa estar indentado**
Tudo que pertence ao else precisa ficar com espaço antes.
- ⊘ **O else é opcional**



Alternativa Moderna: match-case



Sintaxe Limpa

Alternativa para substituir longas sequências de if, elif e else.



Seletor de Padrões

Funciona como um seletor que compara uma variável contra diversos valores fixos.



Caso Padrão (_)

Utiliza o caractere underline para capturar qualquer valor não especificado anteriormente.

 Introduzido no Python 3.10



```
status = 404
```

```
match status:
```

```
    case 200: print("Sucesso")
```

```
    case 404: print("Não encontrado")
```

```
    case _: print("Erro desconhecido")
```




Estruturas de Repetição: for



Iteração Definida

O loop **for** percorre cada item de uma sequência (lista, texto ou intervalo).



Sequências e Coleções

Ideal para processar listas de dados ou executar um bloco um número fixo de vezes.



Automação Eficiente

Elimina a necessidade de escrever o mesmo código manualmente para múltiplos valores.



```
# 0 for repete o bloco de código para cada número  
gerado pelo range  
for numero in range(5):  
  
# Resultado: Cada fruta impressa em uma linha  
    print(numero)
```




Estruturas de Repetição: while

Loop while

Executa repetidamente um bloco de código enquanto uma condição lógica for verdadeira.

Comando break

Força a interrupção imediata do loop, saindo da estrutura de repetição antes da condição falhar.

Comando continue

Ignora o restante do código na iteração atual e salta diretamente para a próxima verificação do loop.

```
  
# Exemplo de controle de fluxo  
  
contador = 0  
  
while contador < 5:  
    contador += 1  
  
    if contador == 3:  
        continue # Pula o número 3  
  
    print(f"Número: {contador}")
```

 **Atenção:** Sempre atualize a variável de controle para evitar loops infinitos.



Modelo Mental de Execução

Memória RAM

Quando o script roda, as variáveis são guardadas na **memória temporária**. Se o programa fecha, a memória é limpa.

Fluxo em Tempo Real

O interpretador Python lê o código de cima para baixo. Se encontrar um erro, a execução **para imediatamente** naquela linha.

01



Código Fonte (.py)

Suas instruções escritas.



02



Interpretador

Traduz linha por linha para o PC.



03



Saída / Terminal

O resultado visível do processamento.



Ficou com alguma dúvida?

Escreva ela nos comentários da aula!

Abertura

✓ Concluir

♡ Favoritar

☆ Avaliar

Informações

Comentários



Escreva sua dúvida aqui ou deixe um feedback da aula! :)|



Cancelar

Comentar



Gostou da aula? Deixe um feedback!

✓ Concluir

♥ Favoritar

☆ Avaliar



Obs: Não se esqueça de clicar em "concluir" a aula e avaliar utilizando as estrelinhas

Avaliar conteúdo

Queremos saber a sua opinião sobre o conteúdo! Por favor, avalie utilizando as estrelas abaixo e, se desejar, escreva um comentário.



Escreva um comentário...

Cancelar

Enviar avaliação



 Jornada
de Dados 



TRILHA_PYTHON